

Mathematical Formulation of Learning and Sampling for Extreme Ground-Motion Events

ExtremeEventFlows

Contents

1	Overview	1
2	The physical event sequence	1
2.1	Variable-length marked point process	1
2.2	Latent-Gaussian transition kernel	2
2.3	Time-increment survival	2
3	Ground-motion model	2
3.1	Feature construction	2
3.2	Multi-output lognormal ground-motion model	3
4	Joint physical (“original”) density	3
5	Rare-event target	3
5.1	Performance function	3
5.2	Smoothed indicator and unnormalized target	4
6	Autoregressive normalizing-flow proposal	4
6.1	Structure	4
6.2	Recurrent state update and stopping	5
7	Learning problem	5
7.1	Objective: forward KL / weighted maximum likelihood	5
7.2	Pretraining	6
7.3	Progressive threshold schedule	6
7.4	Diagnostics	6
8	Sample generation summary	6
9	Rare-event probability estimation	7
9.1	Naive Monte Carlo	7
9.2	Flow-only importance sampling	7
9.3	Defensive-mixture importance sampling	7

1 Overview

This document gives the mathematical description of the physical simulation model, the rare-event target density, the trainable autoregressive normalizing-flow proposal, and the estimators used to learn that proposal and to estimate rare-event probabilities. It mirrors the implementation in `extreme_event_flows/`: `source.py`, `ground_motion.py`, `application.py`, `target.py`, `flow.py`, `training.py`, and `estimators.py`.

The overall goal is to estimate the probability of a rare, extreme event (e.g. exceeding an intensity-measure threshold at a site over an exposure window) under a physical seismic-source and ground-motion model, using importance sampling with a learned proposal that concentrates mass on the failure region.

2 The physical event sequence

2.1 Variable-length marked point process

The seismic source is modeled as a variable-length sequence of events $\mathcal{X}_N = (\mathbf{s}_1, \dots, \mathbf{s}_N)$ arriving on $[0, T]$, where T is the exposure time. Each event vector $\mathbf{s}_i$ collects an arbitrary number of physical source parameters, always including an inter-event time increment Δt_i so that the i -th event occurs at time

$$t_i = \sum_{j=1}^i \Delta t_j.$$

In the reference example,

$$\mathbf{s}_i = (\Delta t_i, M_i, x_i, y_i, d_i),$$

i.e. time increment, magnitude, and 3-D location, but the framework supports any named set of scalar event variables (`source.py: GaussianLatentMarkovKernel`).

The process is Markov in the event history: the next event depends only on the immediately preceding event. The joint density of a sequence with exactly N events inside $[0, T]$ is

$$p(\mathcal{X}_N) = \prod_{i=1}^N p(\mathbf{s}_i \mid \mathbf{s}_{i-1}) \mathbb{P}(\Delta t_{N+1} > T - t_N \mid \mathbf{s}_N), \quad (1)$$

where the last factor is the survival probability that no further event occurs before the exposure horizon T is reached, and $\mathbf{s}_0$ denotes a reference (initial) state.

2.2 Latent-Gaussian transition kernel

Each physical coordinate $s_{i,j}$ of event i is related to a latent Gaussian coordinate $z_{i,j}$ by an invertible scalar transform T_j ,

$$s_{i,j} = T_j(z_{i,j}), \quad j = 1, \dots, d, \quad (2)$$

implemented as monotone bijections in `transforms.py` (e.g. identity, softplus/exponential maps enforcing positivity or bounds, each with a tractable Jacobian $\log|T'_j(z)|$).

Conditional on the previous physical event $\mathbf{s}_{i-1}$, the latent vector $\mathbf{z}_i = (z_{i,1}, \dots, z_{i,d})$ is jointly Gaussian,

$$\mathbf{z}_i \mid \mathbf{s}_{i-1} \sim \mathcal{N}(\boldsymbol{\mu}_i, LL^\top), \quad \boldsymbol{\mu}_i = \boldsymbol{\mu}_0 + A \left(\frac{\mathbf{s}_{i-1} - \mathbf{s}_{\text{ref}}}{\boldsymbol{\sigma}_{\text{state}}} \right), \quad (3)$$

where $\boldsymbol{\mu}_0 \in \mathbb{R}^d$ is a base mean, $A \in \mathbb{R}^{d \times d}$ is a transition matrix creating Markov dependence on every previous-event coordinate, L is a lower-triangular Cholesky factor giving a fully coupled covariance $LL^\top$ across the coordinates of the current event, and $\mathbf{s}_{\text{ref}}, \boldsymbol{\sigma}_{\text{state}}$ are reference values and scales that non-dimensionalize the conditioning feature.

The physical event density follows by the change-of-variables formula,

$$p(\mathbf{s}_i | \mathbf{s}_{i-1}) = \mathcal{N}(\mathbf{z}_i; \boldsymbol{\mu}_i, LL^\top) \prod_{j=1}^d \left| \frac{dz_{i,j}}{ds_{i,j}} \right|, \quad \mathbf{z}_i = (T_1^{-1}(s_{i,1}), \dots, T_d^{-1}(s_{i,d})). \quad (4)$$

2.3 Time-increment survival

Because the time increment Δt is one coordinate of $\mathbf{s}_i$ with its own marginal Gaussian latent coordinate (mean $\mu_i^{(\Delta t)}$, variance $\sigma_{(\Delta t)}^2 = \sum_k L_{(\Delta t),k}^2$ from row (Δt) of L), the survival probability that no event occurs in the remaining exposure window $T - t_N$ is

$$\mathbb{P}(\Delta t_{N+1} > r | \mathbf{s}_N) = \Phi\left(-\frac{T_{\Delta t}^{-1}(r) - \mu_{N+1}^{(\Delta t)}}{\sigma_{(\Delta t)}}\right), \quad r = T - t_N, \quad (5)$$

with Φ the standard normal CDF (`log_ndtr` in the code), and the degenerate case $r \leq$ the transform's lower bound returns log-survival 0.

Sampling the sequence (`source.py: sample_sequence`) proceeds by repeatedly drawing $\mathbf{z}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, LL^\top)$, mapping to physical units, and accepting the event only while $t_i \leq T$; the process terminates the first time a proposed event time exceeds T .

3 Ground-motion model

3.1 Feature construction

For each accepted source event i and a fixed site, a feature map $\mathbf{f}_i = f(\mathbf{s}_i, \text{site})$ is built (`ground_motion.py: GroundMotionFeatureBuilder`). The reference implementation uses

$$\mathbf{f}_i = (1, M_i, \log(R_i + R_0), \log(V_{s30}/V_{s30}^{\text{ref}}), \dots), \quad (6)$$

where $R_i = \|(x_i, y_i, d_i) - (x_{\text{site}}, y_{\text{site}}, d_{\text{site}})\|_2$ is the 3-D source-to-site distance, R_0 is a small offset avoiding a singularity at zero distance, and additional source/site scalar fields may be appended.

3.2 Multi-output lognormal ground-motion model

For K intensity-measure outputs (e.g. PGA, $SA(0.2\text{ s})$, $SA(1.0\text{ s})$), a linear model in log space with correlated residuals gives, for event i ,

$$\boldsymbol{\mu}_i^{(Y)} = B\mathbf{f}_i, \quad \ln \mathbf{Y}_i = \boldsymbol{\mu}_i^{(Y)} + L_Y \boldsymbol{\epsilon}_i, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(0, I_K), \quad (7)$$

where $B \in \mathbb{R}^{K \times \dim(\mathbf{f})}$ are per-output regression coefficients and L_Y is a lower-triangular Cholesky factor of the within-event residual covariance, coupling the K outputs (e.g. PGA and spectral accelerations are correlated within the same event). The residual vector $\boldsymbol{\epsilon}_i$ is a standard normal latent variable exactly as in a normalizing flow, with density

$$p(\boldsymbol{\epsilon}_i) = \prod_{k=1}^K \mathcal{N}(\epsilon_{i,k}; 0, 1). \quad (8)$$

One latent residual is added per ground-motion output; the flow automatically grows to include it (`ground_motion.py: MultiOutputGaussianGroundMotionModel`).

4 Joint physical (“original”) density

Composing the source and ground-motion models gives the joint density of a complete simulated sample $x = (\mathcal{X}_N, \{\mathbf{Y}_i\}_{i=1}^N)$,

$$p(x) = \underbrace{p(\mathcal{X}_N)}_{\text{Eq. (1)}} \times \underbrace{\prod_{i=1}^N p(\boldsymbol{\epsilon}_i)}_{\text{Eq. (8)}}, \quad (9)$$

computed in `application.py`: `log.original.density` as the sum of a source log-density and a ground-motion (residual) log-density.

5 Rare-event target

5.1 Performance function

A scalar performance function $g(x)$ maps a complete sample to a real number such that the rare/failure event is $F = \{x : g(x) \geq 0\}$ (`performance.py`). For a joint ground-motion threshold condition with per-output thresholds γ_k , define the per-event, per-output log margin

$$r_{i,k} = \log\left(\frac{Y_{i,k}}{\gamma_k}\right), \quad (10)$$

an event score aggregating across outputs $k = 1, \dots, K$,

$$\text{event score}_i = \begin{cases} \min_k r_{i,k} & \text{mode} = \text{“all” (joint exceedance)} \\ \max_k r_{i,k} & \text{mode} = \text{“any” (union exceedance)} \\ r_{i,(m)} & \text{mode} = \text{“}k\text{-of-}n\text{”, } m\text{-th largest margin,} \end{cases} \quad (11)$$

and the sequence-level performance as the maximum over events,

$$g(x) = \max_{i=1, \dots, N} \text{event score}_i. \quad (12)$$

The failure event is then $F = \{x : g(x) \geq 0\}$, i.e. some event in the sequence exceeds the threshold condition. General system-level conditions can be supplied as an arbitrary callable of the full sample.

5.2 Smoothed indicator and unnormalized target

Direct use of the indicator $\mathbb{1}_F(x)$ is not differentiable, so a smooth penalty is used in place of a hard indicator during training (`target.py`: `PenaltyFunction`),

$$\rho_\alpha(x) = \exp(-\alpha \text{ReLU}(0 - g(x))) \iff \log \rho_\alpha(x) = -\alpha \text{ReLU}(-g(x)), \quad (13)$$

with steepness $\alpha > 0$. As $\alpha \rightarrow \infty$, $\rho_\alpha(x) \rightarrow \mathbb{1}_F(x)$; $\rho_\alpha(x) = 1$ whenever $g(x) \geq 0$ and decays exponentially for $g(x) < 0$.

The unnormalized rare-event target density is

$$h(x) = p(x) \rho_\alpha(x), \quad \log h(x) = \log p(x) + \log \rho_\alpha(x), \quad (14)$$

computed by `target.py: RareEventTargetDensity`. The (unknown) normalizing constant of h is closely related to the failure probability of interest,

$$\mathbb{P}(F) = \mathbb{E}_p[\mathbb{K}_F(X)] = \int \mathbb{K}_F(x) p(x) dx \approx \int \rho_\alpha(x) p(x) dx = \int h(x) dx \quad (15)$$

in the limit $\alpha \rightarrow \infty$; during training α is annealed upward (via a progressive threshold schedule, Section 7.3) so that h increasingly concentrates on F .

6 Autoregressive normalizing-flow proposal

6.1 Structure

The trainable proposal $q_\theta(x)$ is an event-by-event autoregressive normalizing flow (`flow.py: AutoregressiveEventFlow`) that mirrors the generative structure of $p(x)$ but replaces every conditional Gaussian by a learned Gaussian whose mean and scale are outputs of a neural network conditioned on the full sequence history through a recurrent hidden state.

Within event i , variables are ordered

$$v_i = (\Delta t_i, \text{remaining source variables}, \epsilon_i) \in \mathbb{R}^m, \quad m = |\text{source vars}| + K, \quad (16)$$

and are generated autoregressively *within* the event and *across* events: variable $v_{i,j}$ is conditioned on the recurrent hidden state h_{i-1} (summarizing all previous events), the current/remaining exposure time, and all previously generated raw variables $v_{i,1:j-1}$ of the same event,

$$(\mu_{i,j}(\theta), \log \sigma_{i,j}(\theta)) = \text{MLP}_j(h_{i-1}, t_{i-1}/T, (T - t_{i-1})/T, v_{i,1:j-1}), \quad (17)$$

implemented as one small conditional MLP per variable position (`ConditionalMLP`), with $\log \sigma$ squashed into $[\log \sigma_{\min}, \log \sigma_{\max}]$ via a sigmoid for numerical stability. The raw (latent) variable is drawn

$$z_{i,j} \sim \mathcal{N}(0, 1), \quad \tilde{v}_{i,j} = \mu_{i,j}(\theta) + \sigma_{i,j}(\theta) z_{i,j}, \quad (18)$$

and mapped through the same physical transform T_j used by the source model (identity for residual variables, since they are already standard Gaussian in the physical model):

$$v_{i,j} = T_j(\tilde{v}_{i,j}). \quad (19)$$

The event-level flow log-density (change of variables) is

$$\log q_\theta(v_i | h_{i-1}) = \sum_{j=1}^m \left[\mathcal{N}(\tilde{v}_{i,j}; \mu_{i,j}, \sigma_{i,j}^2) - \log |T'_j(\tilde{v}_{i,j})| \right]. \quad (20)$$

6.2 Recurrent state update and stopping

After event i is generated, the hidden state is updated by a GRU cell consuming the raw event vector and normalized event time,

$$h_i = \text{GRUCell}([\tilde{v}_i, t_i/T], h_{i-1}), \quad h_0 = h_{\text{init}} \text{ (learned)}. \quad (21)$$

Sequence generation stops the first time a proposed event time $t_{i-1} + \Delta t_i$ exceeds T ; as in the physical model, the flow's log-density accrues a matching stopping log-probability using the same

Gaussian time-increment head evaluated as a normal log-survival function (Eq. (5), with μ, σ from Eq. (17) instead of the physical transition). The complete sequence log-density under the proposal is

$$\log q_\theta(x) = \sum_{i=1}^N \log q_\theta(v_i | h_{i-1}) + \log \mathbb{P}_\theta(\Delta t_{N+1} > T - t_N | h_N). \quad (22)$$

This is a valid joint density over variable-length sequences: it is a triangular (autoregressive) transform of an underlying standard-normal base sample, so q_θ is normalized and samples/densities are both tractable (`sample_one` / `log_prob_one`).

7 Learning problem

7.1 Objective: forward KL / weighted maximum likelihood

The proposal is trained to approximate the (self-normalized) rare-event target $h(x)/Z$, $Z = \int h$, by minimizing the forward Kullback–Leibler divergence

$$\theta^\star = \arg \min_{\theta} D_{\text{KL}}\left(\frac{h}{Z} \parallel q_\theta\right) = \arg \max_{\theta} \mathbb{E}_{x \sim h/Z} [\log q_\theta(x)] + \text{const}. \quad (23)$$

Since samples from h/Z are not directly available, a self-normalized importance-sampling / cross-entropy estimator is used (`estimators.py: cross_entropy_update`, `training.py: defensive_mixture_update`). Draws are taken from a *defensive mixture* proposal that mixes the current flow with the original physical model,

$$r_\theta(x) = (1 - \varepsilon) q_\theta(x) + \varepsilon p(x), \quad \varepsilon \in (0, 1), \quad (24)$$

which guarantees non-vanishing support under the physical model even while q_θ is still adapting. For M samples $x^{(1)}, \dots, x^{(M)} \sim r_\theta$, tempered self-normalized importance weights are formed,

$$w^{(m)} = \frac{(h(x^{(m)})/r_\theta(x^{(m)}))^{1/\tau}}{\sum_{m'=1}^M (h(x^{(m')})/r_\theta(x^{(m')}))^{1/\tau}} = \text{softmax}_m \left(\frac{\log h(x^{(m)}) - \log r_\theta(x^{(m)})}{\tau} \right), \quad (25)$$

with temperature $\tau > 0$ smoothing the weight distribution, and the weighted cross-entropy loss minimized by stochastic gradient (Adam) is

$$\mathcal{L}(\theta) = - \sum_{m=1}^M w^{(m)} \log q_\theta(x^{(m)}), \quad (26)$$

which is an unbiased (in the large- M limit) stochastic estimate of the gradient of the KL objective in Eq. (23). Weights are computed under `torch.no_grad()` (self-normalized IS is treated as fixed) while only $\log q_\theta(x^{(m)})$ carries gradients.

7.2 Pretraining

Before any rare-event adaptation, the flow is warm-started by ordinary maximum likelihood on samples from the original physical model p (`training.py: pretrain_on_original_model`),

$$\theta_0 = \arg \min_{\theta} -\frac{1}{M} \sum_{m=1}^M \log q_\theta(x^{(m)}), \quad x^{(m)} \stackrel{\text{iid}}{\sim} p. \quad (27)$$

This matches $q_\theta \approx p$ before any rare-event penalty is applied, providing a stable starting point for cross-entropy adaptation.

7.3 Progressive threshold schedule

Directly training against a very rare threshold γ from initialization is unstable (near-zero effective sample size). Instead, thresholds $\gamma_1 < \gamma_2 < \dots < \gamma_L$ are visited in increasing order (`training.py: ProgressiveThresholdTrainer.train_schedule`), each paired with a penalty steepness α_ℓ ; at each stage the flow and Adam optimizer state from the previous, easier threshold are retained as the initialization,

$$\theta_\ell^{(0)} = \theta_{\ell-1}^*, \quad \theta_\ell^* = \arg \min_{\theta} \mathcal{L}_\ell(\theta), \quad (28)$$

where $\mathcal{L}_\ell$ uses target $h_\ell(x) = p(x)\rho_{\alpha_\ell}(x)$ with threshold γ_ℓ substituted into the performance function. This continuation strategy lets a proposal trained for an easier (more probable) threshold serve as a good initialization for a rarer one, analogous to sequential/adaptive importance sampling and cross-entropy methods for rare events.

7.4 Diagnostics

Every update reports importance-weight diagnostics used to monitor learning health: the maximum normalized weight $\max_m w^{(m)}$, the weight (Shannon) entropy $-\sum_m w^{(m)} \log w^{(m)}$, and the weighted effective sample size

$$\text{ESS} = \frac{1}{\sum_{m=1}^M (w^{(m)})^2}, \quad (29)$$

which should remain a non-trivial fraction of M for a well-adapted proposal.

8 Sample generation summary

Generating one complete simulated sample proceeds as follows.

1. **Initialize** time $t_0 = 0$ and the source state at a reference event $\mathbf{s}_0$ (physical model) or recurrent hidden state h_0 (flow).
2. **Loop over events** $i = 1, 2, \dots$:
 - (a) Draw the latent Gaussian vector for the next event conditional on the previous state (Eq. (3) for p , or Eq. (17) autoregressively coordinate-by-coordinate for q_θ), including the time increment Δt_i first.
 - (b) Map to physical units via the invertible transforms T_j .
 - (c) If $t_{i-1} + \Delta t_i > T$: stop, record the stopping (survival) log-probability, and terminate the sequence with $N = i - 1$ events.
 - (d) Otherwise accept event i , set $t_i = t_{i-1} + \Delta t_i$, build ground-motion features $\mathbf{f}_i$ from the event and site, draw standard-normal residuals ϵ_i (or their flow equivalents) and compute $\mathbf{Y}_i$ via Eq. (7), and update the conditioning state/hidden state for the next event.
3. **Return** the full sample $x = (\mathcal{X}_N, \{\mathbf{Y}_i\})$ together with its log-density (Eq. (9) under p , or Eq. (22) under q_θ).

Because q_θ shares the same variable ordering, transforms, and autoregressive/Markov factorization structure as p , it can generate sequences of arbitrary (and different) length N while remaining absolutely continuous with respect to p almost everywhere, which is what makes it usable as an importance-sampling proposal.

9 Rare-event probability estimation

9.1 Naive Monte Carlo

Direct simulation from the physical model gives an unbiased but typically very high-variance estimator for rare F (`estimators.py: estimate_probability_naive_monte_carlo`),

$$\hat{P}_{\text{MC}} = \frac{1}{M} \sum_{m=1}^M \mathbb{1}_F(x^{(m)}), \quad x^{(m)} \stackrel{\text{iid}}{\sim} p, \quad (30)$$

with a Wilson binomial confidence interval, appropriate when the exceedance count may be small or zero.

9.2 Flow-only importance sampling

Using the trained flow directly as an importance proposal (`estimate_probability_flow_only`),

$$\hat{P}_q = \frac{1}{M} \sum_{m=1}^M \mathbb{1}_F(x^{(m)}) \frac{p(x^{(m)})}{q_\theta(x^{(m)})}, \quad x^{(m)} \stackrel{\text{iid}}{\sim} q_\theta. \quad (31)$$

This is unbiased provided q_θ 's support covers F , but is used mainly as a diagnostic since a poorly-adapted or mode-collapsed flow can miss support entirely.

9.3 Defensive-mixture importance sampling

The safeguarded, final estimator draws from the same defensive mixture used in training (Eq. (24)),

$$\hat{P}_r = \frac{1}{M} \sum_{m=1}^M \mathbb{1}_F(x^{(m)}) \frac{p(x^{(m)})}{r_\theta(x^{(m)})}, \quad x^{(m)} \sim r_\theta, \quad (32)$$

which guarantees a bounded importance weight $p(x)/r_\theta(x) \leq p(x)/(\varepsilon p(x)) = 1/\varepsilon$ whenever the sample is drawn from the physical component, preventing the unbounded-variance failure mode of a pure learned-proposal estimator. All three estimators return standard error, a confidence interval, and the importance-weight effective sample size as diagnostics of estimator reliability.